

EDUCAÇÃO PROFISSIONAL PAULISTA

ROTEIRO DE ATIVIDADE PRÁTICA — ESTUDANTES

Semana 16 — SGBD Relacional

Disciplina: Modelagem e Desenvolvimento de Banco de Dados

Curso: Técnico em Desenvolvimento de Sistemas

Público-alvo: Estudantes do Ensino Médio Técnico

Carga horária: 3 aulas de 50 minutos

Ambiente das aulas práticas: OneCompiler com SQLite

Aula 1: teoria | Aulas 2 e 3: prática

Material adaptado a partir dos conteúdos da Semana 16, preservando os conceitos centrais e substituindo comandos exclusivos do MySQL por alternativas compatíveis com SQLite quando necessário.

1. Identificação

Nome	Turma	Data
_____	_____	_____
_____	_____	_____

2. Contexto

A FastTech está organizando seu banco de dados de vendas. O sistema precisa relacionar clientes, produtos e pedidos, acelerar algumas consultas, controlar alterações com segurança e facilitar relatórios usados com frequência. Nesta atividade, você construirá uma base pequena e aplicará esses conceitos em SQLite pelo OneCompiler.

Importante: A atividade original foi proposta em MySQL. Nesta versão, os comandos foram adaptados para SQLite porque será utilizado o OneCompiler no laboratório. O objetivo conceitual permanece o mesmo.

3. Objetivos

- Criar tabelas relacionadas com chaves primárias e estrangeiras.
- Executar uma consulta com JOIN.
- Criar e verificar índices.
- Testar ROLLBACK e COMMIT.
- Criar uma VIEW e consultá-la.
- Simular uma view materializada armazenando o resultado em uma tabela.
- Registrar evidências das etapas realizadas.

4. Organização das aulas práticas

Aula	Tempo	Atividade
2	50 min	Tabelas, dados, JOIN, índices e EXPLAIN QUERY PLAN.
3	50 min	ROLLBACK, COMMIT, VIEW e simulação materializada.

5. Preparação do OneCompiler

1. Abra o OneCompiler no navegador.
2. Selecione a linguagem/ambiente SQLite.
3. Apague o exemplo inicial do editor.
4. Cole ou digite os comandos de cada etapa na ordem indicada.
5. Clique em Run para executar e observe a área de resultados.

Regra da atividade: Mantenha PRAGMA foreign_keys = ON no início do script. Ele ativa a verificação das chaves estrangeiras no SQLite.

AULA 2 — Parte 1: chaves, JOIN e índices

Etapa 1 — Criar as tabelas

Cole o script abaixo. Ele começa removendo versões antigas das tabelas para facilitar novos testes e depois cria a estrutura do banco.

```
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS relatorio_vendas_materializada;
DROP VIEW IF EXISTS relatorio_vendas;
DROP TABLE IF EXISTS itens_pedido;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS produtos;
DROP TABLE IF EXISTS clientes;

CREATE TABLE clientes (
  id_cliente INTEGER PRIMARY KEY AUTOINCREMENT,
  nome TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL
);

CREATE TABLE produtos (
  id_produto INTEGER PRIMARY KEY AUTOINCREMENT,
  nome TEXT NOT NULL,
  categoria TEXT NOT NULL,
  preco NUMERIC NOT NULL CHECK (preco >= 0)
);

CREATE TABLE pedidos (
  id_pedido INTEGER PRIMARY KEY AUTOINCREMENT,
  data_pedido TEXT NOT NULL,
  id_cliente INTEGER NOT NULL,
  status TEXT NOT NULL DEFAULT 'Pendente',
  FOREIGN KEY (id_cliente)
    REFERENCES clientes(id_cliente)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

CREATE TABLE itens_pedido (
  id_item INTEGER PRIMARY KEY AUTOINCREMENT,
  id_pedido INTEGER NOT NULL,
  id_produto INTEGER NOT NULL,
  quantidade INTEGER NOT NULL CHECK (quantidade > 0),
  FOREIGN KEY (id_pedido)
    REFERENCES pedidos(id_pedido)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (id_produto)
    REFERENCES produtos(id_produto)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Depois de executar, responda:

- Qual campo é a chave primária de clientes? `Id_cliente`.
- Qual campo de pedidos aponta para clientes? `Id_cliente`.
- Por que quantidade possui CHECK (`quantidade > 0`)? Para impedir que a quantidade seja zero ou negativa.

Etapa 2 — Inserir dados

```
INSERT INTO clientes (nome, email) VALUES
('Ana Souza', 'ana@fasttech.com'),
('Bruno Lima', 'bruno@fasttech.com'),
('Carla Mendes', 'carla@fasttech.com');

INSERT INTO produtos (nome, categoria, preco) VALUES
('Notebook', 'Informática', 3500.00),
('Mouse', 'Informática', 120.00),
('Cadeira Gamer', 'Móveis', 950.00),
('Monitor 24', 'Informática', 900.00);

INSERT INTO pedidos (data_pedido, id_cliente, status) VALUES
('2026-08-10', 1, 'Pago'),
('2026-08-10', 2, 'Pendente'),
('2026-08-11', 3, 'Pago');

INSERT INTO itens_pedido (id_pedido, id_produto, quantidade) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 4, 1),
(3, 3, 1);
```

Execute e, em seguida, teste:

```
SELECT * FROM clientes;
SELECT * FROM produtos;
SELECT * FROM pedidos;
SELECT * FROM itens_pedido;
```

Etapa 3 — Relacionar as tabelas com JOIN

Agora consulte informações que estão espalhadas por quatro tabelas. Leia cada JOIN com atenção.

```
SELECT
    pedidos.id_pedido,
    clientes.nome AS cliente,
    produtos.nome AS produto,
    itens_pedido.quantidade
FROM pedidos
JOIN clientes
    ON pedidos.id_cliente = clientes.id_cliente
JOIN itens_pedido
    ON pedidos.id_pedido = itens_pedido.id_pedido
JOIN produtos
    ON itens_pedido.id_produto = produtos.id_produto
ORDER BY pedidos.id_pedido, itens_pedido.id_item;
```

Quantas linhas foram exibidas? 4 linhas.

Por que o pedido 1 pode aparecer mais de uma vez?

Porque o pedido 1 possui dois produtos diferentes.

Etapa 4 — Criar índices

Crie um índice simples e um índice composto na tabela produtos.

```
CREATE INDEX idx_produtos_categoria
ON produtos(categoria);

CREATE INDEX idx_produtos_categoria_preco
ON produtos(categoria, preco);
```

Liste os índices:

```
PRAGMA index_list('produtos');
```

Analise o plano de uma consulta:

```
EXPLAIN QUERY PLAN
SELECT *
FROM produtos
WHERE categoria = 'Informática'
AND preco > 1000;
```

Observação: Como nossa tabela possui poucos registros, não tente medir “velocidade” pelo tempo. O objetivo é verificar como o SQLite planeja a consulta e reconhecer a presença dos índices.

EVIDÊNCIA 1 — Estrutura criada

Insira um print do OneCompiler mostrando a execução da criação das tabelas ou os dados consultados após a criação.

The screenshot shows the OneCompiler interface with a file named 'queries.sql'. The code in the editor includes:

```
1 PRAGMA foreign_keys=ON;
2
3 DROP TABLE IF EXISTS produtos;
4 DROP TABLE IF EXISTS clientes;
5 DROP TABLE IF EXISTS fornecedores;
6 DROP TABLE IF EXISTS pedidos;
7
8 CREATE TABLE clientes (
9   id_cliente INTEGER PRIMARY KEY,
10  nome TEXT,
11  email TEXT,
12 );
```

The output window shows the result of a query, displaying a table with 4 columns: id_cliente, nome, email, and telefone. The data row shows:

id_cliente	nome	email	telefone
3	Carla Mendes	carla@fasttech.com	

Explique brevemente o que este print comprova:

EVIDÊNCIA 2 — Resultado do JOIN

Insira um print do resultado da consulta com JOIN mostrando cliente, produto e quantidade.

INSIRA O PRINT AQUI

Explique brevemente o que este print comprova:

Print das tabelas criadas.

EVIDÊNCIA 3 — Índices / EXPLAIN QUERY PLAN

Insira um print de PRAGMA index_list ou EXPLAIN QUERY PLAN.

INSIRA

OneCompiler

Upgrade

AI

SQLite

Run

Save

Login

queries.sql

1 PRAGMA foreign_keys = ON;

2

3 DROP TABLE IF EXISTS itens_pedido;

4 DROP TABLE IF EXISTS pedidos;

5 DROP TABLE IF EXISTS produtos;

6 DROP TABLE IF EXISTS clientes;

7

8 CREATE TABLE clientes (

9 id_cliente INTEGER PRIMARY KEY,

10 nome TEXT NOT NULL,

11 email TEXT UNIQUE NOT NULL

I/O

AI Agent

26 ms | 4.6 MB

STDIN

Input for the program (Optional)

Output

id_cliente	nome	email
1	Ana Souza	ana@fasttech.com
2	Bruno Lima	bruno@fasttech.com
3	Carla Mendes	carla@fasttech.com

Explique brevemente o que este print comprova:

AULA 3 — Parte 2: transações e views

Etapa 5 — Testar ROLLBACK

O objetivo é inserir temporariamente um pedido e depois desfazer a transação.

```
SELECT COUNT(*) AS pedidos_antes
FROM pedidos;

BEGIN TRANSACTION;

INSERT INTO pedidos (data_pedido, id_cliente, status)
VALUES ('2026-08-13', 3, 'Pendente');

SELECT * FROM pedidos;

ROLLBACK;

SELECT COUNT(*) AS pedidos_depois
FROM pedidos;
```

Complete: Antes da transação havia 3 pedidos. Após o ROLLBACK ficaram 3 pedidos.

O pedido temporário permaneceu salvo? () Sim (x) Não

Etapa 6 — Testar COMMIT

Agora faça uma transação semelhante, mas confirme a alteração.

```
BEGIN TRANSACTION;

INSERT INTO pedidos (data_pedido, id_cliente, status)
VALUES ('2026-08-14', 2, 'Pago');

COMMIT;

SELECT * FROM pedidos;
```

O novo pedido permaneceu salvo depois do COMMIT? (x) Sim () Não

Explique a diferença observada entre COMMIT e ROLLBACK:

Commit confirma e salva as alterações feitas na transação. ROLLBACK desfaz as alterações feitas na transação.

Etapa 7 — Criar a view relatorio_vendas

A view reunirá informações de várias tabelas e calculará o total de cada item.

```

DROP VIEW IF EXISTS relatorio_vendas;

CREATE VIEW relatorio_vendas AS
SELECT
    p.id_pedido,
    c.nome AS cliente,
    pr.nome AS produto,
    pr.categoria,
    i.quantidade,
    pr.preco,
    (i.quantidade * pr.preco) AS total_item,
    p.status
FROM pedidos p
JOIN clientes c
    ON p.id_cliente = c.id_cliente
JOIN itens_pedido i
    ON p.id_pedido = i.id_pedido
JOIN produtos pr
    ON i.id_produto = pr.id_produto;

SELECT *
FROM relatorio_vendas;

SELECT *
FROM relatorio_vendas
WHERE preco > 100;

```

Qual vantagem você percebe em consultar `relatorio_vendas` em vez de reescrever todos os JOINS?

A view facilita a consulta porque já deixa os JOINS e cálculos prontos. Assim, não é necessário escrever todo o código novamente.

Etapa 8 — Simular uma view materializada

Nesta atividade, o resultado será copiado para uma tabela. Isso simula o princípio de armazenar fisicamente o resultado de uma consulta.

```

DROP TABLE IF EXISTS relatorio_vendas_materializada;

CREATE TABLE relatorio_vendas_materializada AS
SELECT *
FROM relatorio_vendas
WHERE preco > 100;

SELECT *
FROM relatorio_vendas_materializada;

```

Não confunda: `relatorio_vendas_materializada` é uma tabela criada para simulação. Ela não é uma **MATERIALIZED VIEW** nativa do SQLite.

Para atualizar essa tabela depois de mudanças nos dados, ela deve ser recriada com `DROP TABLE` e `CREATE TABLE ... AS SELECT`.

EVIDÊNCIA 4 — ROLLBACK ou COMMIT

Insira um print demonstrando o resultado de uma das transações.

The screenshot shows the OneCompiler web interface. On the left, a SQL query is written in a text editor:

```

98 EXPLAIN QUERY PLAN
99 SELECT *
100 FROM produtos
101 WHERE categoria = 'Informática'
102 AND preco > 1000;
103
104 CREATE VIEW relatorio_vendas AS
105 SELECT
106     p.id_pedido,
107     c.nome AS cliente,
108     pr.nome AS produto,
  
```

On the right, the 'Output' section displays a table with 4 rows and 4 columns:

1	Notebook	Informática	3500
2	Mouse	Informática	120
3	Cadeira Gamer	Móveis	950
4	Monitor 24	Informática	900

Explique brevemente o que este print comprova:

EVIDÊNCIA 5 — VIEW relatorio_vendas

Insira um print do SELECT executado sobre relatorio_vendas.

The screenshot shows the OneCompiler web interface. On the left, a SQL query is written in a text editor:

```

98 EXPLAIN QUERY PLAN
99 SELECT *
100 FROM produtos
101 WHERE categoria = 'Informática'
102 AND preco > 1000;
103
104 CREATE VIEW relatorio_vendas AS
105 SELECT
106     p.id_pedido,
  
```

On the right, the 'Output' section displays a table with 4 rows and 4 columns:

4	bruno lima	Monitor 24	1
3	Carla Mendes	Cadeira Gamer	1

Explique brevemente o que este print comprova:

9. Questões finais

1. Qual é a função de uma chave primária?

Identificar cada registro de uma tabela de forma única.

2. Qual é a função de uma chave estrangeira?

Relacionar uma tabela com outra.

3. Por que um índice pode melhorar uma consulta?

Porque ajuda o banco de dados a encontrar os dados mais rapidamente.

4. Qual é a diferença entre COMMIT e ROLLBACK?

COMMIT salva as alterações e ROLLBACK desfaz as alterações da transação.

5. Para que serve uma VIEW?

Serve para salvar uma consulta e facilitar o acesso aos dados sem precisar escrever os JOINS novamente.

6. Qual é a diferença entre a view comum e a simulação de view materializada feita nesta atividade?

A VIEW comum executa a consulta quando é acessada. A simulação de view materializada copia o resultado para uma tabela, armazenando os dados fisicamente.

10. Checklist

- ☐ Abri o OneCompiler utilizando SQLite.
- ☐ Mantive PRAGMA foreign_keys = ON.
- ☐ Criei clientes, produtos, pedidos e itens_pedido.
- ☐ Identifiquei PK e FK.
- ☐ Inseri os registros.
- ☐ Executei a consulta com JOIN.
- ☐ Criei um índice simples.
- ☐ Criei um índice composto.
- ☐ Utilizei EXPLAIN QUERY PLAN.
- ☐ Testei ROLLBACK.
- ☐ Testei COMMIT.
- ☐ Criei relatorio_vendas.
- ☐ Consultei a view.
- ☐ Criei a tabela de simulação materializada.
- ☐ Registre os cinco prints solicitados.
- ☐ Respondi às seis questões finais.

11. Critérios de avaliação

Critério	Valor
Criação e organização das tabelas	1,5
PK, FK e integridade referencial	1,5
Inserção e relacionamento dos dados	1,0
JOIN	1,0
Índices e análise da consulta	1,5
Transações / COMMIT / ROLLBACK	1,5
VIEW e simulação materializada	1,0
Evidências e respostas	1,0
TOTAL	10,0

12. Entrega

Entregue o roteiro preenchido com as respostas e os prints solicitados. O professor poderá pedir que você explique oralmente alguns comandos para verificar a compreensão da atividade.